

ECE 622: Modeling and Verification of Embedded Systems (Spring 2026)

Problem Set 4: Prof. Daniel Holcomb

Name: Jaideep Khare and Shu-lin Shih

Problem Set 4 Submission, Date: 04/27/2026

Answer 13.1(a) FALSE

Justification:

$GF\ p$ means that p is true infinitely often. However, this does not mean that p eventually becomes permanently true. A trace may alternate forever between states where p is true and false. In such a trace, $GF\ p$ holds, but $FG\ p$ does not hold.

Counterexample trace:

$p, \neg p, p, \neg p, p, \neg p, \dots$

In this trace, p appears infinitely many times, so $GF\ p$ is true. But there is no point after which p remains true forever, so $FG\ p$ is false.

Answer 13.1(b) TRUE.

Justification:

Gp means that p holds at every position of the trace. Applying another global operator outside it does not strengthen the condition. If p holds globally from the beginning, then at every suffix of the trace, p also holds globally. Therefore, $G(Gp)$ and Gp are equivalent.

Answer 13.2(b) FALSE.

Justification:

There is a counterexample trace where x is present only once and is absent thereafter.

Starting from state a , let x be present in the first step. The machine then reaches state c . After that, let x remain absent forever. Since x is absent forever after reaching c , the machine stays on the path where $y = 1$ is never produced.

Counterexample trace:

$x, \neg x, \neg x, \neg x, \dots$

Along this trace, x occurs initially, but $y = 1$ never occurs afterward. Therefore, the eventual-response requirement is violated.

Answer 13.2(c) TRUE.

Justification:

This formula says that if x is globally true, meaning x is present at every step, then eventually $y = 1$ must occur.

Unlike part 2(b), here we only consider traces where x remains present forever. In such traces, the default transition to state c caused by absence of x never occurs. Therefore, the machine cannot get stuck in the counterexample behavior from part 2(b).

Since x is always present, the machine continues taking the transitions enabled by x , and along this behavior it eventually reaches a transition/state where $y = 1$ is produced.

Answer 14.1(b) None

Justification:

Step 1. We check if B is a type refinement of A .

A has input ports: $\{x\}$

B has input ports: $\{x, w\}$

$P_B \subseteq P_A$ is false.

B is not a type refinement of A .

Step 2. We check if A is a type refinement of B

B has output ports: $\{y\}$

A has output ports: $\{\}$

$Q_B \subseteq Q_A$ is false.

A is not a type refinement of B .

Answer 14.1(d) None

Justification:

Machine A :

Input ports: $P_A = \{x\}$

Output ports: $Q_A = \{y\}$

$x : \mathbb{N}$

$y : \{0, 1\}$

Machine B:

Input ports: $PB = \{x, w\}$

Output ports: $QB = \{y\}$

$x : \{0, 1\}$

$w : \{0, 1\}$

$y : \mathbb{R}$

Since $PA \subseteq PB$ is trivially true, we are checking whether A is a type refinement of B:

Step 1. Input/Output port comparison

$PA \subseteq PB$ is true, since $\{x\} \subseteq \{x, w\}$

$QB \subseteq QA$ is also true, since $\{y\} \subseteq \{y\}$

Step 2. Port type comparison

$\forall x$ of B: $\{0, 1\}$

$\forall x$ of A: $\mathbb{N}$

Since $\{0, 1\}$ is not a subset of $\mathbb{N}$,

A is not a type refinement of B.

Answer 14.6

Yes. We can construct a one-state nondeterministic machine B that simulates A.

The machine B has one state “s” and, for every input value, can nondeterministically output either 1 or 2.

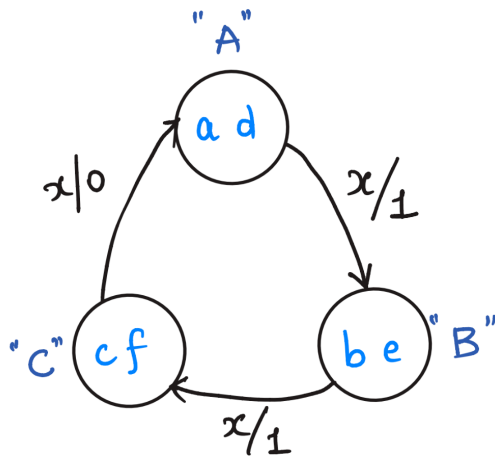
The simulation relation is given by:

$S = \{ (a, s), (b, s), (c, s), (d, s) \}$

Thus, B simulates A because it can match every possible output that A may produce.

Answer 14.7(c)

the required three-state bisimilar machine is:



with bisimulation relation:

$$S = \{ (a, A), (d, A), (b, B), (e, B), (c, C), (f, C) \}$$

This three-state machine is bisimilar to the original six-state machine because every transition in either machine can be matched by the other with the same output, and the resulting states remain related.

Answer 14.8(a) TRUE.

Justification: This is pretty much the same as our solution for 14.6. Yes, A simulates B due to being maximally non-deterministic

Answer 14.8(c) TRUE.

Justification: Since Machine A simulates Machine B,

using Theorem 14.1 (strong language containment) we have:

$$L(B) \subseteq L(A)$$

Which means any LTL property that holds true for B's input/output traces must follow for A as well.